

“You’re on a bicycle with a little motor”: Benefits and Challenges of Using AI Code Assistants

Wendy Mendes
Federal University of Pará
Belém, Pará, Brazil
wendymgaleno@gmail.com

Samara Souza
Federal University of Pará
Belém, Pará, Brazil
samara.souza@icen.ufpa.br

Cleidson R. B. de Souza
Federal University of Pará
Belém, Pará, Brazil
cleidson.desouza@acm.org

ABSTRACT

AI code assistants, such as Tabnine, GitHub CoPilot, and ChatGPT, employ Large Language Models (LLMs) trained on extensive source code and other documents. They receive prompts and generate code suggestions aimed to facilitate programming tasks. Previous research in this field has explored the correctness, complexity, quality, and security of the code suggestions. Software developers’ experiences have been studied in the context of controlled experiments. Based on 14 interviews with software developers, this paper describes the developers’ daily and continuous experiences with AI code assistants, presenting benefits and challenges grounded in actual development work, along with strategies to address these challenges.

KEYWORDS

AI Code Assistants. Developer Experiences. Code Generation

ACM Reference Format:

Wendy Mendes, Samara Souza, and Cleidson R. B. de Souza. 2024. “You’re on a bicycle with a little motor”: Benefits and Challenges of Using AI Code Assistants. In *2024 IEEE/ACM 17th International Conference on Cooperative and Human Aspects of Software Engineering (CHASE ’24)*, April 14–15, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3641822.3641882>

1 INTRODUCTION

In the past few years, the first generation of AI code assistants has been released. Examples of these assistants include Tabnine (released in 2018), GitHub CoPilot (2021), ChatGPT (2022), and Amazon CodeWhisperer (2022) among others. There are both proprietary and open-source (e.g., FauxPilot¹, CodeParrot² and In-Coder [10]) implementations available.

These assistants receive a prompt as an input and generate several tokens as an output, that is, they aim to facilitate the programming task by translating a natural language input (often a comment) or a function signature into a corresponding implementation. In

contrast to “traditional” code completions approaches that only suggest the next token (e.g. Microsoft Intellisense) and that have been around for years in current IDEs, these assistants suggest *sets of tokens* [22]. This is possible because these assistants are trained on hundreds of millions of lines of source code and use Large Language Models (LLMs) to support code-related tasks.

Research in AI code assistants is quickly growing and has focused on different aspects including the benchmarks necessary to evaluate and compare them [7], the correctness [27], complexity [17], quality [16], and security [18, 20] of the generated code, the developers’ experience while using these assistants in controlled studies [2, 22], among other aspects.

In this paper, we are interested in the developers’ experience with AI code assistants. Most of the work regarding this user experience has been conducted in laboratory settings [15], thus focusing on a very limited user experience, mostly focused solely on the tasks being executed. In this paper, we aim to address this limitation by focusing on *the user experience of engineers who are using AI code assistants in their daily software development work*, not in experimental tasks. Accordingly, the contribution of this paper is a set of benefits and challenges of these AI code assistants grounded in real development work. Furthermore, we also present strategies used by software engineers to deal with the challenges associated with AI code assistants. Adopting the metaphor articulated by one of the interviewed software engineers, the advantages can be likened to a small motor on a bicycle, whereas the obstacles parallel the requisite effort of pedalling.

Our results are based on semi-structured interviews conducted with 14 software developers who had been using AI code assistants for, at least, four months at the time of the interview. One of the main benefits of these assistants is to allow software developers to focus on the most complex and strategic aspects of their coding. Meanwhile, their low accuracy is an important challenge that developers try to address by adding more and more context (e.g., providing meaningful names for variables and methods).

The rest of the paper is organized as follows. Section 2 provides a background so that the reader can understand AI code assistants. Section 3 delves into the usage of intelligent code assistants. Section 4 outlines the methodology employed in this study, followed by the presentation of findings in Section 5. In Section 6, we discuss our results in the context of previous work, while Section 7 explores the limitations of our research. Section 8 presents our conclusions and ideas for future work.

2 BACKGROUND

Transformers, an architecture introduced in 2017 [23], play a central role in the advancement of Large Language Models (LLMs),

¹<https://github.com/fauxpilot/fauxpilot>

²<https://huggingface.co/codeparrot/codeparrot>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CHASE ’24, April 14–15, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0533-5/24/04...\$15.00
<https://doi.org/10.1145/3641822.3641882>

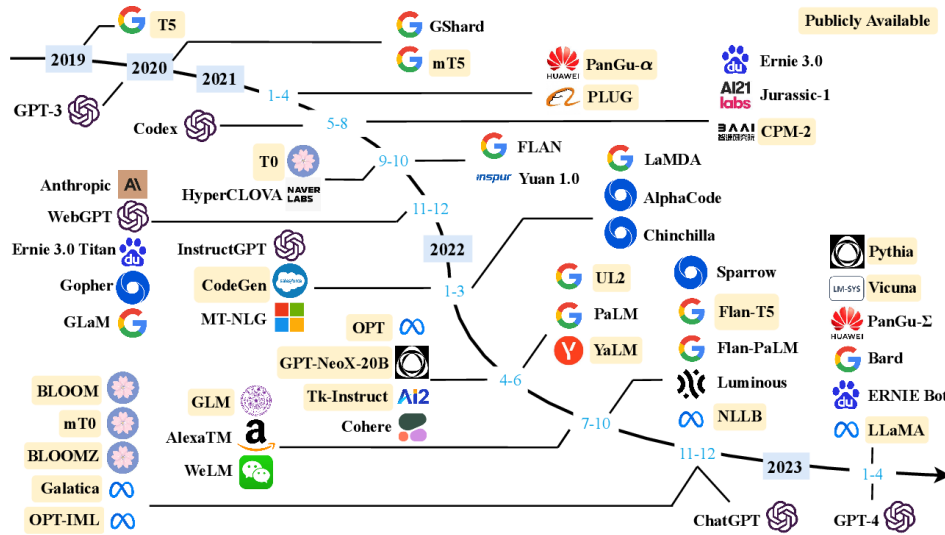


Figure 1: A Timeline of Recent Advances in Large Language Models [28].

such as OpenAI's GPT-3 [9]. Transformers represent a paradigm shift in natural language processing and deep learning, allowing models to understand and generate text based on context and contextual relevance. The most prominent feature of Transformers is the self-adaptive attention mechanism, which enables the model to assign weights to the relevant parts of text sequences dynamically. This contextual attention is essential for capturing long-range relationships and understanding complex nuances in language [23].

Transformer-based models have revolutionized the ability of artificial intelligence (AI) models to understand and produce contextually relevant text. Trained on large sets of text data, these models can answer questions, perform translations, and create coherent and informative content in different contexts, i.e., they excel at natural language processing tasks. This development has profound implications in several areas, from virtual assistants and task automation to improvements in machine-assisted writing and software development.

Figure 1 illustrates the evolution of Large Language Models (LLMs) and provides a compelling visual narrative of the rapid progress in natural language processing in recent years. It shows the emergence of transformative models such as T5, GPT-3, Codex, and GPT-4, highlighting the continuous refinement and innovation in the field. The timeline depicted in the figure serves as a testament to the significant progress made in LLM research and its growing importance in the field of artificial intelligence and language understanding.

LLMs use deep neural network architectures in which successive layers of artificial neurons process textual information [12]. These networks are trained on large amounts of text to learn patterns, grammatical structures, and semantic relationships. The core of how LLMs work is their ability to predict the next word in a sequence, given the previous text in which the word was found. This prediction task is repeated many times to train the model, improving its ability to understand context and generate coherent text [6].

With a significant number of adjustable parameters [2], LLMs such as GPT-3 and Codex (the LLM behind GitHub Copilot) acquire extensive knowledge on various topics, allowing them to interpret questions, create original content, and even simulate the writing style of human authors. This highly customizable text and code generation mechanism enhances the ability of LLMs to produce high-quality text across a range of applications.

In addition to text generation, LLMs can play a crucial role in code generation, assisting programmers with tasks ranging from the simplest to the most complex. Different LLM-based tools that support code generation implement different interaction mechanisms: some tools focus on code completion (e.g., Github Copilot), others allow their users to chat with the underlying LLM (like ChatGPT), and others allow both interaction mechanisms (as recently announced by Amazon CodeWhisperer).

A critical aspect to consider at this stage of the technology development is the accuracy of the results presented by these AI code assistants [17, 27]. As mentioned by Ziegler et al. [29], several authors have thoroughly investigated the quality of code suggestions generated by LLMs and their suitability for use in real software development projects. The accuracy of the generated code is an important consideration, as inaccuracies can lead to security vulnerabilities [18, 20], malfunctions, and other problems that directly affect the quality and reliability of the final software. In general, the ability to evaluate and manage the results produced by assistants is critical to ensuring the quality of the final code. In addition, ethical issues, liability, privacy, and attribution of responsibility for the generated code are also relevant for the software engineers [8]. All of these challenges are fundamental to understanding the effectiveness and acceptance of AI code assistants in software development and should be carefully considered when using these innovative technologies. Some initial studies have explored how AI code assistants can be used by developers. We discuss that in the next section.

3 USING AI CODE ASSISTANTS

With the rise of AI code assistants in recent years, it is important to pay close attention to the benefits that arise from using them as well as the challenges software developers face. Only a few studies have been published on this topic. These studies either focus on specific implementations of code generation tools or GitHub Copilot.

3.1 User Studies of Specific LLM-based code generation tools

Xu et al. [26] created their own AI code assistant, called NL2Code, as an IDE plugin. This tool was evaluated in an experiment with 31 developers in 14 programming tasks “covering a wide range of skills a Python developer may need in practice”. Experiment subjects were either university students or selected from a freelance platform. Their results reveal participants enjoyed interacting with the plugin. Specifically, most users reported having a neutral (15/31; 48.4%) or at least somewhat positive (15/31; 48.4%) experience using the plug-in.

Meanwhile, Jiang et al. [13] examined how programmers utilize GenLine, their LLM-based tool that translates natural language requests into code. In their evaluation, 14 participants with differing levels of front-end coding experience used GenLine to create two small web-based applications. The results suggest that participants felt they had to “learn” how to interact with the tool because of unexpected model responses so that they were able to input “the specific words to say and the specific phrasings to produce the desired output”. Participants also adopted specific strategies when the model did not produce the expected results. For instance, they often rephrased the request, added information, reduced information, and/or made changes to one GenLine parameter.

3.2 User Studies of Github Copilot

Vaithilingam et al. [22] conducted an experiment with 24 students and observed how they implemented three tasks (extracted from Xu et al. [26]) using GitHub Copilot. Their results indicate that the students enjoyed using the tool because it provided a starting point for their work, even when the code it generated was incorrect. They also reported two coping strategies to deal with Copilot’s limitations: “to accept the incorrect suggestion and attempt to repair it” or simply stop using the tool.

Meanwhile, Barke and colleagues [2] studied 20 people (6 professional developers and 14 academic, mostly students) using Copilot in a partially realistic programming task and concluded that: “*interactions with programming assistants are bimodal: in acceleration mode, the programmer knows what to do next and uses Copilot to get there faster; in exploration mode, the programmer is unsure how to proceed and uses Copilot to explore their options*”.

Last, but not least, Bird and colleagues [3] used 3 different data sources to study how GitHub Copilot was being used by software developers in January of 2022. The first data source, public forums, illustrated use cases where Copilot was used as well as concerns with tools (e.g. licensing). Interestingly, users reported issues with interruptions caused by Copilot as well as feeling they had less understanding of how or why the code worked. The second data source was an experiment with 5 expert developers who had to

implement a simple game. Finally, the last source, data focused on the impact of the copilot on developers’ productivity.

Our paper aims to address the limitations of previous studies and focus on the software engineers’ user experience using AI code assistants in their *daily* development tasks in contrast with most studies that study *experimental* tasks.

4 RESEARCH METHODOLOGY

Our study employed a qualitative approach, with a focus on understanding the perceptions and experiences of software developers regarding their real usage of AI code assistants. To collect data, we conducted 14 semi-structured interviews with developers using Github Copilot and/or ChatGPT [9], arguably the most used and discussed tools. Copilot was released in June 2021 [11]. Since its release, it has been integrated into several development platforms, including Visual Studio Code and JetBrains. Importantly, Copilot is powered by OpenAI’s Codex models [7]. These Codex models are refined by fine-tuning the GPT-3 [4] using public Github repositories as learning sources. Meanwhile, ChatGPT is a Natural Language Processing (NLP) system developed by OpenAI that can generate human-like conversations. It is an artificial intelligence technology that can be used to automate conversations and generate more accurate responses [9].

4.1 Data Collection

Our research took place in two rounds, involving different sources. Together, these two data collection stages provided a comprehensive insight into the perceptions and practices related to the real use of intelligent assistants in the context of software development.

In the first round, we identified participants who shared their experiences of using an intelligent assistant tool (Copilot) in their daily lives for software development. At this stage, we employed an approach of analyzing the comments and interactions of Twitter users who expressed either positive or negative opinions about Copilot to gain an in-depth understanding of experiences related to the tool. During this analysis, invitations were sent to potential participants (tweet authors), inviting them to take part in an interview, and as a result, we were able to reach a total of 8 developers. Unfortunately, the developers who answered our invitation tweeted positive aspects about the tool. This is a limitation in our work that we will discuss again later.

In the second phase, we conducted interviews with 6 developers from a locally established software company to further explore the data collected in the first phase. The selection of this company was strategic since the contacts within the company were close to our network. This proximity facilitated access to people and made the invitation to participate in interviews viable. This is a medium-sized company with about 200 employees that develops software for different customers including the local government, and local, national, and international organizations.

We conducted a total of 14 semi-structured interviews. On average, each interview had a duration of 33.57 minutes (53 max and 18 min), being conducted online and recorded with the due authorization of the participants. To provide a more detailed overview of our participants, Table 1 presents specific information about each of them.

Table 1: Demographics of the interviewees. (In the 'Usage Time' column, the format 'X/Y' represents the time using a tool, i.e. where 'X' is the time using Copilot, 'Y' is the time using ChatGPT for code generation).

#	Gender	Role	Age (y.o.)	Exp.(y)	Usage	Duration	Usage Time
P1	M	Software Developer	25y	7y	Copilot/ChatGPT	25m	>6mo
P2	M	Software Developer	19y	1y	Copilot	22m	>6mo
P3	M	Software Developer	32y	12y	Copilot	18m	>6mo
P4	M	Software Developer	23y	4y	Copilot/ChatGPT	34m	>12/5mo
P5	M	Tech Lead	32y	16y	Copilot/ChatGPT	34m	>12/5mo
P6	M	Software Developer	27y	2y	Copilot/ChatGPT	48m	>12/5mo
P7	M	Software Developer	30y	11y	Copilot/ChatGPT	27m	>12/5mo
P8	M	Tech Lead	30y	10y	Copilot/chatGPT	26m	>12/5mo
P9	M	Software Developer	28y	7y	ChatGPT	27m	>6mo
P10	M	Software Developer	31y	7y	ChatGPT	47m	>6mo
P11	M	Software Developer	21y	1y	ChatGPT	34m	>6mo
P12	M	Innovation leader	34y	16y	ChatGPT	53m	>6mo
P13	M	Data Engineer	28y	3y	ChatGPT	37m	>4mo
P14	M	Software Developer	32y	8y	ChatGPT	38m	>6mo
Average			28 y	7.5 y	-	33.57 m	-
Standard deviation			4.52 y	5.05 y	-	10.34 m	-
Máx			34 y	16 y	-	53 m	-
Min			19 y	1 y	-	18 m	-

It is important to highlight in Table 1 that most of our Twitter interviewees have been using AI code assistants since their launch, while the other interviewees have been using them for a couple of months. We emphasize this here to illustrate the fact that we have early adopters [19] in our sample. These people are willing to take risks and have the ability to absorb and understand new ideas. They play a crucial role in the process of diffusion of technologies as they serve as opinion leaders [19].

We created an interview guide to help us during the data collection. The topics presented in our guide and covered in our interviews include: (i) gathering demographic data (How long have you been in this role?), programming languages used (Which languages do you work with?), (ii) exploration of professionals' perceptions about intelligent assistants (What is your perception of this tool?), benefits (What kind of assistance do you seek from the tool?), and challenges (Have you encountered any limitations while using the tool?), (iii) investigation of the relationship between the interviewees and their companies with the use of the tool (Have you already shared that you use the tool with other people?) and (iv) interviewees' expectations about potential changes in the way the software is developed with the use of AI, (Do you think a tool like GitHub Copilot/ChatGPT will change the way software is developed in the future? Yes, no? why?). The combination of these questions served as a basis for capturing information on the interviewees' perceptions and practical usage of AI code assistants in their software development efforts.

4.2 Data Analysis

All interviews were transcribed and subjected to analysis through an open coding process [21]. This process was conducted by the authors using spreadsheets. Initially, one of the authors took on the task of creating an initial list of codes and identifying keywords or

relevant codes in the interview data. As the process advanced, the list was reviewed and refined by the other authors. Based on this initial list of codes, we conducted repeated collaborative analyses to identify similarities between the codes to develop higher-order codes or categories. During the process, we also recognized the need to refine the identified categories. Finally, we conducted a thorough review of the categories and made modifications to their nomenclature when necessary to ensure that they represented the information found in the interview data more clearly.

Any conflicts were resolved through discussion and consensus meetings, and a third author validated this coding process. The aim of this analysis was to understand how software developers carry out their software development activities using AI code assistants. Furthermore, we sought to identify the specific benefits and challenges that these professionals experienced when incorporating these tools into their daily work.

5 FINDINGS

Figure 2 presents a graphic representation of our results, i.e., the *benefits* observed by software developers when using AI code assistants, the *challenges* they faced, and the *strategies* they used to address these challenges. In the following sections, we will describe each one of these aspects in detail. Before doing so, we describe some general information about the usage of the AI code assistants by our interviewees.

In addition to the benefits, challenges and strategies, Figure 2 also presents some arrows among these elements. The dotted arrow from "Interruption" to "Faster development" highlights that interruptions can act as obstacles, impairing the fluidity and speed of development. Additionally, the relationship between "Add more context" and "Faster development" suggests that the quest for a deeper understanding of the problem which is necessary to improve

the prompt and increase the accuracy of the tool output, while beneficial, may have a negative impact on development speed. The arrow points towards prioritizing "Faster development" through a "Refined focus", indicating a more targeted approach by software developers who can focus on the most complex and/or strategic aspects of the coding task. The arrow also suggests that the pursuit of greater accuracy, represented by "Low Accuracy", is associated with the "Add More Context" strategy, implying that adding more context to the tool may alleviate this challenge.

5.1 Overall Usage

As for the programming language used with the code assistants, we noticed that most of our interviewees used with Java, C#, Python, and JavaScript, while a few of them used it with other languages like PHP, TypeScript, Dart, and GoLang. This variation is interesting since the correctness of Copilot's suggestions is different depending on the programming language used [17]. Some developers even use these tools to be able to handle multiple programming languages. In this case, P4 mentioned: *"I already mentioned the example of FOR, and to this day, I get confused about the syntax I'll have to use for that language, and Copilot always helps me with that. In my specific case, I work with multiple languages at the same time, and never remembering how the syntax for something specific is, it helps a lot."*

One of the key questions addressed during the interviews with software professionals pertains to *when they use AI in their development activities*. We had mixed results with about half of our respondents using them all the time. For instance, P5 mentioned: *"All the time activated, if it's deactivated, I go crazy. (...) I write code, and I look over in the corner, in its case, it's in the bottom right corner of the IDE, to see if it's thinking, if it has any suggestions, what's happening. But it's activated all the time."* Meanwhile, the other half of the interviewees use the AI code assistant differently, like P2: *"It's not turned on all the time, but I usually turn it on when there's a part that's more boilerplate, you know?"*

5.2 Benefits

In this section, we will delve into the various benefits perceived by developers when introducing intelligent assistance tools into their daily software development workflows.

5.2.1 Faster development. One benefit highlighted by the interviewees when using AI code assistants in their workflow is to expedite the code creation process, resulting in time savings and the ability to complete tasks more quickly. For instance, P3 reports that these tools not only speed up the development process but also make the work more seamless, giving a sense of enhanced productivity³. He compared his experience using an AI code assistant to being on a bicycle with a motor (*"I think it increased, but if you look at it metrically, analyzing how much I do per sprint, I see that I'm still the same. It's more in the sense that when you're on a bicycle with a little motor, you keep pedalling, but there's a little motor there"*), P3. Another interviewee, P13, also mentioned something similar: *"[my productivity] did increase, especially when I don't know anything about the subject or when I just want to write a quick function that I already know how to do and just want to speed up the process."*

³Note that we are reporting what the interviewees said without discussing whether the concepts they used, e.g., productivity, are accurate or not.

Furthermore, interviewees used AI code assistants as a support resource to complement specific code creation tasks. For instance, P9 describes: *"I typically use it as code autocomplete. So, when I'm writing there, for example, I create a variable called 'user,' and Copilot completes 'user' with a name and a CPF"*⁴. In the same way, P3 emphasizes *"I use it more as an autocomplete, but I have used it to help me complete code I didn't know. Usually, I start doing something, and I hope it analyzes what I'm doing and provides me with suggestions"*. This specific benefit was primarily observed among participants who use GitHub Copilot, thanks to its ability to provide contextual code suggestions that auto-complete lines of code.

In addition to saving developers' time by using it as an autocomplete, we observed another strategic use of AI, especially ChatGPT, as a tool that contributes by avoiding the need to search for additional information through other means, such as internet search engines. *"In Google, I used to waste a lot of time clicking on links to see if they had the information I was looking for. But with chatGPT, you ask the questions, and it already comes back with a solution"*, P11. Similarly, P10 mentioned that he used to spend a lot of time using Stack Overflow (SO). However, this opinion is not widespread. For instance, P4 mentioned a more social view of SO: *"At least I see Stack Overflow more as a place to search and ask questions, but also as a social network. So, now and then, I'm around there, trying to answer a question or two. During development, sometimes there are things that don't work or an error that we need to look up somewhere, find an example of how someone solved it, and many of these things Copilot doesn't have."*

5.2.2 Refined focus. Faster development time allows software developers to spend their time on more complex and strategic aspects, i.e., developers can focus on more important aspects. P3 clearly expresses this when argues: *"I think it accelerates [my work], we stop doing more operational things, and think more about intelligent things, so I think the work will evolve into one of [...] I'm going to think more about the business and less about the way I'm going to implement it. These types of tools help us think more about the business and less about the repetitive things that every business has to do"*.

Another interviewee, P5, mentioned something similar: *"I can clearly notice an improvement in my productivity, no doubt. Even if it's, as I mentioned, trivial things like defining a loop, I can do that in just a few seconds. But I don't want to do it; I want the AI to complete it quickly so I can focus on the real problem I have to work on"* (emphasis added).

5.2.3 Improved Code. Many participants responded that they use AI assistants as a tool to enhance the quality of the code they develop. One of the interviewees mentioned using these assistants to rewrite code (*"I've finished writing a class or a part of the code. I ask chatGPT to [...] rewrite this in a better way or give more meaningful names to my variables, or I can ask all of this at once, and it already provides it for us"*), P4.

P6 posed a similar argument about using the AI code assistant for simplifying their code (*"I also learned to use these tools a lot to try to improve my code. Because, as I entered the IT field along with this new wave of artificial intelligence, I feel like I still have a deficit in writing cleaner, more functional, and more concise code. So, I really*

⁴CPF is a unique identified in Brazil, like social security number in the US

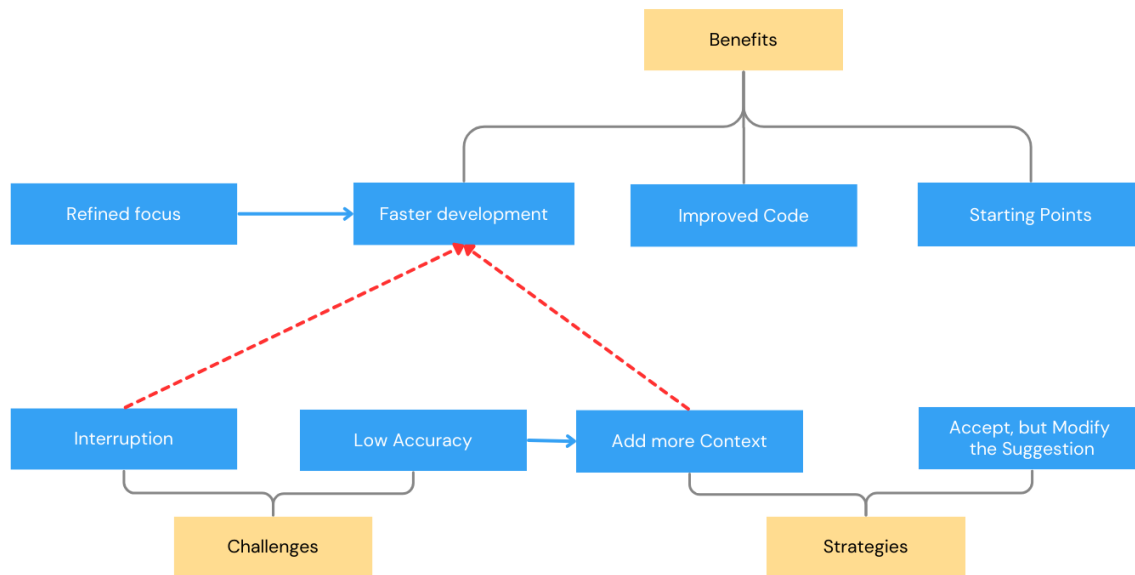


Figure 2: Visual Representation of our Findings

like using these tools to rewrite the code I write”), P6. We noticed that for this type of use, these assistants become allies in technical excellence, allowing developers to correct errors and optimize their code. As we will discuss later, the correctness limitations of these assistants are a major concern.

5.2.4 Starting Points. While many of our interviewees use AI code assistants to complement code snippets in programming languages or tasks they have experience with, others turn to these assistants to provide them with a starting point for their tasks. In this case, developers use code assistants to help them begin a task for which they have no or limited knowledge (e.g., the structure or a particular API). As illustrated by P1, (“So, there were times when I had to create a function that performed a certain action, and I didn’t know exactly how it should be; I only had a rough idea. The tool was there and provided everything I needed”), Participant P2 also shared: “But if I’m not very familiar with the well-known API I’m working with, I sometimes let it do the work and tell me how it’s initially implemented”.

An even clearer example is shared by P13 when describing how the results from chatGPT were combined with a Google search and other information-seeking activities: “It gave me the code to create the feature, but I think I had the wrong variable. However, since it gave me the name of the class that I could use to create the feature, I just took the class, put it in Google, and then I could see the parameters of this class to figure it out. So, it gave a kickstart to speed up the development.” We can conclude that by seeking assistance from AI and receiving initial insights, developers can begin their tasks even when working with new, unfamiliar programming languages and/or APIs.

These starting points can also be seen as learning opportunities for the developers. P14, for instance, mentioned: “What has been really helping me with this tool is precisely the process of trying to understand something that I’m not currently grasping about what

a certain code is supposed to be doing, what might be coming from it, or which part of the code a specific error could be associated with.” Another example is illustrated by interviewee P11: “For example, when I want to optimize a piece of code or clarify a doubt. Then it can provide me with an alternative solution to what I want to achieve.”

5.3 Challenges

Although we have observed that Copilot and ChatGPT are helpful for software developers, it is important to note that there are times when their use can become disadvantageous, negatively impacting the software development and the developers’ user experience.

5.3.1 Low Accuracy. Most interviewees acknowledged that, although the code assistants have the capability to generate code, not all suggestions have a good level of accuracy. In specific situations, imprecise or poor suggestions are returned, requiring them to refine their input into the tool, as confirmed by P9 (“I ask for a certain thing, and normally it gives me something quite wrong, and then I refine it”). Interviewee P11 also reports (“(...) The first time you ask, it doesn’t give you all the information. You have to keep feeding it with more information, asking more specific things”).

For the interviewees, the accuracy of the suggestions is not only associated with their correctness but also with an important temporal aspect of the programming languages: versioning. Specifically, P7 mentioned the following “I also noticed that the suggestions it [the code assistant] gives aren’t necessarily up-to-date. So, for example, I work with PHP, and PHP has evolved from version 5 to 8.2. If I ask for code now, it won’t give me code using version 8.2. I have to specifically request it to update the code, update the suggestion with the newest version, or something like that. This sometimes bothers me because when I’m working on a project with a newer version, I want to use the features of that function.” This quote also suggests that the AI code assistants are also limited regarding their training dataset and, as a consequence, their users might perceive them.

P4 explained that one way to address this type of problem is conducting a code review, (“if what it suggested is critical and doesn’t work, in case I let it pass, we also work with code review. So, for example, when I submit a pull request, someone will probably take a look at it and give a heads up”).

Finally, it is important to mention two aspects regarding the continued use of coding assistants. First, at least one software developer, P6, mentioned his initial tendency to accept the assistants’ suggestions, even when those suggestions contained errors or did not fully align with the project’s requirements. He only realized this later: “I’ve had a bad experience because I’ve uploaded code that I thought was good, but in production, it was unworkable, you know? And there wasn’t this treatment to check if something works in a local environment but definitely doesn’t work in production”. And, second, another software developer, P7, identified a context in which he does not accept the suggestions: database queries. According to him: “One thing that Copilot often suggests a lot but I almost never accept is regarding database queries. It constructs queries for the database. Usually, at this point, Copilot always makes mistakes, and I have to keep correcting it.”. Arguably, these are examples of situations that can only be observed over time, i.e., since P6 and P7 were early adopters of code assistants, they already had enough time to recognize the limitations of this technology.

5.3.2 Interruption. One issue mentioned by the interviewees was the constant volume of suggestions generated by the assistant, which, in some cases, can lead to distractions. Developers noted that Copilot continues to provide suggestions persistently, even when they wish to work without interruptions temporarily, as evidenced by P1 (“What I don’t like, for example, yesterday I was writing CSS, and I wrote a line, and I jumped to the next one, and I didn’t want to accept the suggestion, but it kept suggesting, which bothered me a little because I didn’t want suggestions at that moment, but it kept auto-completing”).

Our interviewees also observed that the code suggestions occur even when a software developer is at the beginning of the code creation process, where there is not much context available, and the suggestions can be less precise: (“I believe that when I’m starting a project from scratch, I tend to keep it turned off because, as it still has very little context of what you’re doing, it tends to provide a lot of suggestions, which kind of hinders my thought process”), P2. In this case, future work should assess whether these interruptions might end up nullifying the benefit of faster software development.

5.4 Strategies

During the interviews, developers also shared some of the strategies they employ to address the challenges related to the use of code assistants in software development.

5.4.1 Add more Context. In general, to address the issue of low accuracy (see section 5.3.1), developers increase the context provided to the tools. This is explained by P8: (“When I’m doing something very large and in a hurry, it doesn’t work at all because it needs more context to implement anything, you know?”).

Adding more context includes (1) creating a very specific prompt (“So there’s a step today that I often take before talking to ChatGPT, which is creating my prompt, creating my question. So, I open a

notepad, think about what I’m going to put in the structure, and then I copy and paste it into the chat”), P12; or (2) being very descriptive when creating variables and functions, i.e., giving them meaningful names (“I understand that the more descriptive you can be, the better Copilot is. In the sense that, if you name the function exactly what you expect from that function, it will provide a solution or proposal that is closer to that”), P6. These approaches aim to maximize the suggestions generated by intelligent assistants while minimizing inaccurate suggestions. It should be noted that the effort required by software developers to add more context to the prompt might end up nullifying the benefit of faster software development.

5.4.2 Accept, but Modify the Suggestion. When the previous strategy, focused on increasing the accuracy of AI suggestions, does not achieve the desired results, developers turn to another alternative - directly correcting or enhancing the code suggestions. In other words, developers perform a preliminary analysis of the suggested code snippet and if it aligns with what they are creating, they accept the suggested code and modify it to better fit their context. P2 reports this when he says: “I usually try to check something else before accepting the suggestions”. P13 complements by saying that he always makes changes to the code snippet (“I always make some modification, even changing the name of a variable to make it easier or to adapt to the context, but I always end up having to make some changes”). Similarly, P4 reports that he accepts the suggestions, but analyzes them first (“I accept the suggestion, but I always accept and confirm if the code is really functional or not, if it aligns with my code quality and accomplishes what I want to do, it implements what it suggested”). In summary, developers review and adjust the suggested code snippets to ensure that the resulting code meets project specifications and their quality standards. This means not blindly accepting all suggestions but considering each of them on a case-by-case basis.

6 DISCUSSION

Our results presented the *benefits* and *challenges* associated with the use of AI code assistants. These *challenges* are addressed by *strategies* adopted by the interviewed software developers. We also identified how some of these challenges and strategies might end up reducing the identified benefits. All these results are based on the *continuous* use of these tools after a minimum of four months (P13) and with several developers (P4-P8) using the code assistants for over a year (see Table 1). This is the main contribution of this paper and contrasts with previous studies (see section 3) that are based on the benefits and challenges reported after using these tools in limited tasks during controlled experiments.

Some of the *benefits* we identified have been reported in the literature before (faster development [2] and starting points [22, 25]). However, to the best of our knowledge, the other benefits - refined focus (a consequence of faster development), and improved code - have not. Arguably, they can only be learned after continuous use of these tools. More importantly, the perception that developers can benefit from AI code assistants generating improved code *might* be a concern since studies about the correctness of these assistants indicate a situation far from ideal regarding the correctness of the suggestions (e.g., as low as 27% for JavaScript [17]).

Meanwhile, the main *challenge* of these AI code assistants (low accuracy) has been recognized as a major limitation of any LLM-based tool (e.g., see Bang's et al. discussion of ChatGPT [1]). However, our results indicate that in the context of software development, this also means wrong versioning of the suggested code (as mentioned by P7 when discussing PHP code), and the acknowledgment of the limitations of code assistants for particular scenarios (database queries as indicated by P7).

Interruption, the second challenge we reported, was only briefly mentioned in previous studies [3], while we present empirical evidence about the context when it happens (early programming stages) as well as the implications associated with it. In this case, a potential negative impact on development time. These results suggest that AI code assistant tools should better calibrate their model that identifies opportunities for code suggestions [29] because these models seem to be making suggestions "too early", i.e., based on incomplete or not enough prompts.

Our results also indicate that our interviewees are aware of the AI code assistants' limitations and maintain critical vigilance of them. For instance, P7 has to specifically request suggestions in the newer version of the programming language and, most of the time, validate these suggestions before incorporating them into their code. In other words, software developers created *strategies* to deal with the AI assistants' challenges: they either work around the limitations of the tools by fixing the suggestions or simply avoid using them. These strategies are similar to the ones reported by Xu et al. [26]. However, it is important to mention that adding more context is somewhat worrisome since the longer the context, the higher the likelihood of the LLM (including the ones used in AI code assistants) generating incorrect answers [14]. Furthermore, adding more context means more effort from software developers, which can jeopardize the main benefit of these tools - faster development. Future work should explore how chain-of-thought or other prompting techniques [24] could be somehow integrated with AI code assistants.

Last, but not least, it is important to mention that the most common results we identified are somewhat expected: faster development - these tools have been designed with this goal in mind - and low accuracy, a limitation of most LLM-based tools [1]. Therefore, it is not surprising that the results we found "in the wild" are similar to the results identified during the experiments described in previous studies.

7 LIMITATIONS

Our study constitutes an initial analysis of how intelligent assistants are being used by software developers in their real software development tasks. As such, there are some limitations to be considered. While we obtained data from two different sources, one of which included interviewees from Twitter authors working at different companies, and the second one focused on participants from a single company, our sample cannot be considered fully representative of all developers using AI code assistants in software development. In addition, all interviewed developers were from Brazil, but there are no specific reasons to find them different from developers in other countries. It is also important to note that our study has not yet reached data saturation. Future studies should aim

to further explore and confirm these findings with a more diverse and extensive group of software developers.

Furthermore, it is crucial to acknowledge that our participant pool exclusively comprises male software developers. This limitation may overlook potential gender differences in programming tool preferences, as demonstrated in previous studies [5]. Recognizing this gap, we emphasize our commitment to addressing this limitation in future research endeavors. The goal is to actively seek the inclusion of highly diverse participants, including women developers, to enhance the study's robustness and ensure a more comprehensive understanding of the research topic.

Additionally, critically examining potential researchers' biases is essential, as they hold the potential to inadvertently shape multiple facets of the study, ranging from the formulation of interview questions to the subsequent analysis and interpretation of data. Despite efforts to maintain an impartial stance, admitting and addressing the potential impact of the researcher's biases remains a pivotal consideration in the accurate interpretation of the findings presented in this research.

8 CONCLUSIONS AND FUTURE WORK

This study provided an understanding of software engineers' real use of AI code assistants in their work environments. The identified benefits, such as faster development, refined focus, code improvement, and easier starting points, highlight the transformative potential of these tools in software development. However, we are not oblivious to the inherent challenges, such as low accuracy and unwanted interruptions, which emerge as significant trade-offs. The strategies adopted by our interviewees to overcome these challenges also are an important result of this research.

The contribution of this paper is to contextualize these elements in real work environments, highlighting aspects that are often overlooked in controlled laboratory settings. In addition to providing valuable insights into the experiences of the software developers, we also present concrete strategies for overcoming the challenges associated with AI code assistants. In doing so, we hope to enrich the dialogue around these tools and promote a more holistic understanding of their usefulness and impact on the software development process.

In this way, this research has shown that the contributions resulting from the implementation of intelligent assistants in software development are notable, attesting to the positive impact and ease provided to developers who use these tools. However, it is imperative to recognize the challenges identified, which, far from nullifying the contributions, indicate that we are in the early stages of their evolution, representing a transformation in the software development process. It should be noted that AI code assistants are redefining this process with their technology, but it is crucial to understand that the effort and active participation of developers are still required as if they were on a bicycle with a small engine, where they must actively contribute to drive progress.

For future work, several avenues merit exploration to enhance our understanding of the utilization of AI code assistants in software development. Firstly, conducting additional interviews with a diverse pool of developers will provide deeper insights into their experiences and perspectives, enriching our findings. Furthermore,

undertaking a comprehensive survey across a broader spectrum of practitioners will serve to corroborate and generalize the identified benefits and challenges, ensuring a more robust and inclusive assessment. Additionally, a crucial focus should be directed toward a thorough examination of alternative AI code assistant tools, aiming to compare and contrast their functionalities, strengths, and limitations. As we discussed in section 3, most studies focus on Github Copilot. Such a comparative analysis will contribute to a comprehensive evaluation of available tools, offering valuable guidance for developers in selecting the most suitable AI code assistant for their specific needs.

ACKNOWLEDGMENTS

We would like to thank the reviewers for their valuable comments. In addition, we want to thank CNPq (grant numbers 420406/2023-9 and 442779/2023-2) for partially funding this research. We also extend our appreciation to CAPES for their support through scholarships.

REFERENCES

- [1] Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, Quyet V. Do, Yan Xu, and Pascale Fung. 2023. A Multitask, Multilingual, Multimodal Evaluation of ChatGPT on Reasoning, Hallucination, and Interactivity. *arXiv:2302.04023* [cs.CL]
- [2] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.
- [3] Christian Bird, Denae Ford, Thomas Zimmermann, Nicole Forsgren, Eirini Kalliamvakou, Travis Lowdermilk, and Idan Gazit. 2023. Taking Flight with Copilot: Early Insights and Opportunities of AI-Powered Pair-Programming Tools. *Queue* 20, 6 (jan 2023), 35–57. <https://doi.org/10.1145/3582083>
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [5] Margaret Burnett, Scott D Fleming, Shamsi Iqbal, Gina Venolia, Vidya Rajaram, Umer Farooq, Valentina Grigoreanu, and Mary Czerwinski. 2010. Gender differences and programming environments: across programming populations. In *Proceedings of the 2010 ACM-IEEE international symposium on empirical software engineering and measurement*. 1–10.
- [6] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Kaijie Zhu, Hao Chen, Linyi Yang, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2023. A survey on evaluation of large language models. *arXiv preprint arXiv:2307.03109* (2023).
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgren Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [8] Cleidson R. B. de Souza, Geema Rodriguez-Pérez, Manaal Basha, Dongwook Yoon, and Ivan Beschastnikh. 2024. The fine balance between helping with your job and taking it: AI Code Assistants come to the fore (To appear). *IEEE Software* (2024), 6.
- [9] Jianyang Deng and Yijia Lin. 2022. The benefits and challenges of ChatGPT: An overview. *Frontiers in Computing and Intelligent Systems* 2, 2 (2022), 81–83.
- [10] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen tau Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. *arXiv:2204.05999* [cs.SE]
- [11] Nat Friedman. 2021. Introducing GitHub Copilot: your AI pair programmer. *URL https://github.blog/2021-06-29-introducing-github-copilot-ai-pair-programmer* (2021).
- [12] Muhammad Usman Hadi, R Qureshi, A Shah, M Irfan, A Zafar, MB Shaikh, N Akhtar, J Wu, and S Mirjalili. 2023. A survey on large language models: Applications, challenges, limitations, and practical usage. *TechRxiv* (2023).
- [13] Ellen Jiang, Edwin Toh, Alejandra Molina, Kristen Olson, Claire Kayacik, Aaron Donsbach, Carrie J Cai, and Michael Terry. 2022. Discovering the syntax and strategies of natural language programming with generative language models. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–19.
- [14] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. *arXiv:2307.03172* [cs.CL]
- [15] JOSEPH E. MCGRATH. 1995. METHODOLOGY MATTERS: DOING RESEARCH IN THE BEHAVIORAL AND SOCIAL SCIENCES. In *Readings in Human-Computer Interaction*, RONALD M. BAECKER, JONATHAN GRUDIN, WILLIAM A.S. BUX-TON, and SAUL GREENBERG (Eds.). Morgan Kaufmann, 152–169. <https://doi.org/10.1016/B978-0-08-051574-8.50019-4>
- [16] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Zhen Ming (Jack) Jiang. 2023. GitHub Copilot AI pair programmer: Asset or Liability? *Journal of Systems and Software* 203 (2023), 111734. <https://doi.org/10.1016/j.jss.2023.111734>
- [17] Nhan Nguyen and Sarah Nadi. 2022. An Empirical Evaluation of GitHub Copilot’s Code Suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories (Pittsburgh, Pennsylvania) (MSR ’22)*. Association for Computing Machinery, New York, NY, USA, 1–5. <https://doi.org/10.1145/3524842.3528470>
- [18] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. 754–768. <https://doi.org/10.1109/SP46214.2022.9833571>
- [19] Everett M. Rogers. 2003. *Diffusion of innovations* (5th ed.). Free Press, New York, NY [u.a.], 576 pages.
- [20] Mohammed Latif Siddiq and Joanna C. S. Santos. 2022. SecurityEval Dataset: Mining Vulnerability Examples to Evaluate Machine Learning-Based Code Generation Techniques. In *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security (Singapore) (MSR4P&S 2022)*. Association for Computing Machinery, New York, NY, USA, 29–33. <https://doi.org/10.1145/3549035.3561184>
- [21] Anselm Strauss and Juliet Corbin. 1998. Basics of qualitative research techniques. (1998).
- [22] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*. 1–7.
- [23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [24] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv:2201.11903* [cs.CL]
- [25] Justin D. Weisz, Michael Muller, Stephanie Houde, John Richards, Steven I. Ross, Fernando Martinez, Mayank Agarwal, and Kartik Talamadupula. 2021. Perfection Not Required? Human-AI Partnerships in Code Translation. In *26th International Conference on Intelligent User Interfaces* (College Station, TX, USA) (IUI ’21). Association for Computing Machinery, New York, NY, USA, 402–412. <https://doi.org/10.1145/3397481.3450656>
- [26] Frank F. Xu, Bogdan Vasilescu, and Graham Neubig. 2022. In-IDE Code Generation from Natural Language: Promise and Challenges. *ACM Trans. Softw. Eng. Methodol.* 31, 2, Article 29 (mar 2022), 47 pages. <https://doi.org/10.1145/3487569>
- [27] Burak Yetistiren, Isik Ozsoy, and Eray Tuzun. 2022. Assessing the Quality of GitHub Copilot’s Code Generation. In *Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering (Singapore) (PROMISE 2022)*. Association for Computing Machinery, New York, NY, USA, 62–71. <https://doi.org/10.1145/3558489.3559072>
- [28] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).
- [29] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 21–29.